Niech nastanie wiosna

Tymoteusz Huszcza

TL;DR, co wiemy?

- czym jest backend
- co robi backend
- jak robi backend
- podstawowe zabezpieczenia backenda
- czym jest spring boot

Czego nie wiemy?

- nie wiem, jak zrobić projekt?

Od czego by tu zająć?

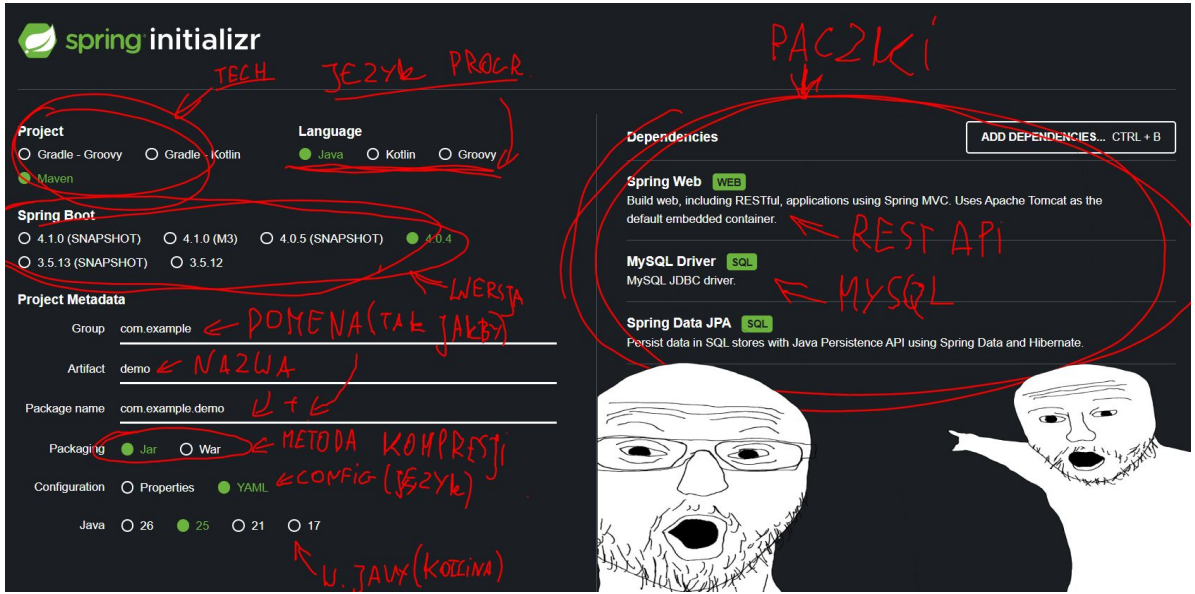
pobrać jakieś ide, np. intellij idea

pobrać odpowiednią wersję javy

jakaś baza danych, np. mysql przez xampp/mysql server/docker

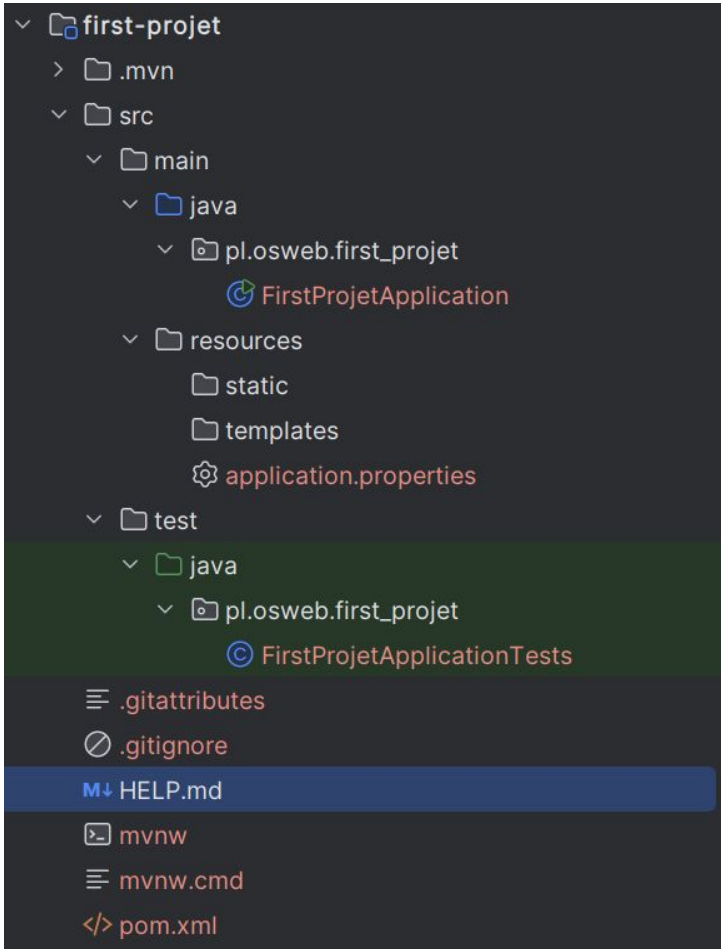
tworzymy prokekt na: https://start.spring.io/

Krok 1: stworzyć projekt



Daje nam to tego potworka:

Użyliśmy Maven’a, wersja - nie znaleziono,
grupa: pl.osweb
nazwa: first-project
pakiety w jar
konfigurację w properties
java: 25



Krok 2: skonfigurować bazę danych

ustawiamy zmienne aplikacji (środowiskowe) - np. baze danyc w application.properties
src/main/resorces/
!bazę musimy stworzyć!

application.properties

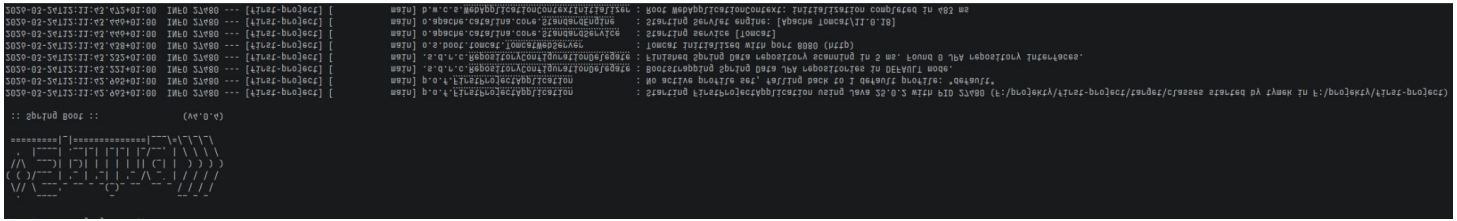
```
spring.application.name=first-projet
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/[nazwa bazy]
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
```

Krok 3: odpalamy projekt

Odpalamy w terminalu, by pobrać zależności przy pomocy:
Win: .\mvnw.cmd spring-boot:run

Mac/Linux: ./mvnw spring-boot:run



Działanie

Stworzymy teraz 6 pakietów w src/main/java/[nazwa-projektu]:

- entities
- services
- controllers
- interfaces
- enums
- dtos

Krok 1: stworzenie jakiegoś Entity

Przedstawienie obiektu z bazy danych w kodzie. Tworząc entity, Java Spring Boot sama stworzy nam nową tabelę w bazie. Stworzymy UserEntity w pakiecie entities. Stworzymy również enum UserRole w enums.

@Entity - flaga do oznaczania entity

@Table - flaga do ustawiania jak będzie prezentować się tabela w bazie

@Id - pole oznaczone jako primary key, id

@GeneratedValue - automatycznie tworzona wartość, może być liczbowa i cyfrowa, u nas uuid

@Column - przedstawienie kolumny i jej wartości w bazie

@Enumerated - oznaczenie pola jako enum, może przyjąć ordinal lub string, oznaczający czy w bazie będzie reprezentowany jako numer czy string

```java
@Entity
@Table(name = "user")
public class UserEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @Column(nullable = false, unique = true)  no usages
    private String username;

    @Column(nullable = false)  no usages
    private String password;

    @Column(nullable = false)  no usages
    @Enumerated(EnumType.STRING)
    private UserRole role = UserRole.USER;

    //gettery, settery
}
```

```java
public enum UserRole {  no usages
    ADMIN,  no usages
    USER  no usages
}
```

Krok 2: tworzymy repo i DTO

Interfejs który przetłumaczy entity na normalny obiekt, tworzymy go w pakiecie interfaces i w nim nowy interface używający interface JpaRepository<T, ID>
DTO odpowiada za przedstawienie danych przychodzących / wychodzących z API
@Repository - flaga do oznaczania repo
JpaRepository<T, ID> - typ repozytorium, przyjmujący Entity i typ ID, u nas String

```
public record UserDTO (String username, String password) { }
```

```
@Repository  no usages
public interface UserRepo extends JpaRepository<UserEntity, String> {
}
```

Krok 3: Serwis

Odpowiadają za logikę modułu.
Tworzymy pakiet services, a w nim nową klasę używającą flagę Service.
@Service - flaga do oznaczania serwisu

@Autowired - Spring Boot będzie nam przekazywał wartość automatycznie do konstruktora

```java
@Service
public class UserService {
    final UserRepo userRepo;  6 usages

    @Autowired
    public UserService(UserRepo userRepo) { this.userRepo = userRepo; }

    public UserEntity create(UserDTO newUser){  no usages
        UserEntity user = new UserEntity();
        user.setUsername(newUser.username());
        user.setPassword(newUser.password());
        return userRepo.save(user);
    }

    public List<UserEntity> getAll() { return userRepo.findAll(); }

    public Optional<UserEntity> getById(String id) { return userRepo.findById(id); }

    public UserEntity updateUser(String id, UserDTO updatedUser){  no usages
        Optional<UserEntity> oldUser = userRepo.findById(id);
        if(oldUser.isPresent()){
            UserEntity user = oldUser.get();
            user.setUsername(updatedUser.username());
            user.setPassword(updatedUser.password());
            user.setRole(updatedUser.role());
            return user;
        }else{
            throw new RuntimeException("User not found.");
        }
    }

    public void delete(String id) { userRepo.deleteById(id); }
}
```

Krok 4: Kontroler

Tutaj tworzymy konkretne endpointy. W pakiecie controllers tworzymy nową klasę UserController.

@RestController - flaga do oznaczenia kontrolera

@RequestMapping - domyślna ścieżka do endpointu

@GetMapping - oznaczenie metody GET, można też oznaczyć konkretne zakończenie endpointu

@PostMapping - podobnie dla metody POST

@PutMapping - tak samo, ale PUT,

@DeleteMapping - tak samo, ale DELETE.

@PathVariable - zmienna z url. w url zmienne oznaczamy między klamerkami ({}) w endpoincie, po czym pobieramy ja w funkcji właśnie z tą flagą.

@RequestBody - zawartość requesta

ResponseEntity - obiekt z funkcjami zmieniającymi nam odpowiedź z serwera

```java
@RestController
@RequestMapping("/user")
public class UserController {
    private final UserService userService;  6 usages

    @Autowired
    public UserController(UserService userService) { this.userService = userService; }

    @PostMapping
    public ResponseEntity<UserEntity> create(@RequestBody UserDTO user){
        UserEntity newUser = userService.create(user);
        return ResponseEntity.ok(newUser);
    }

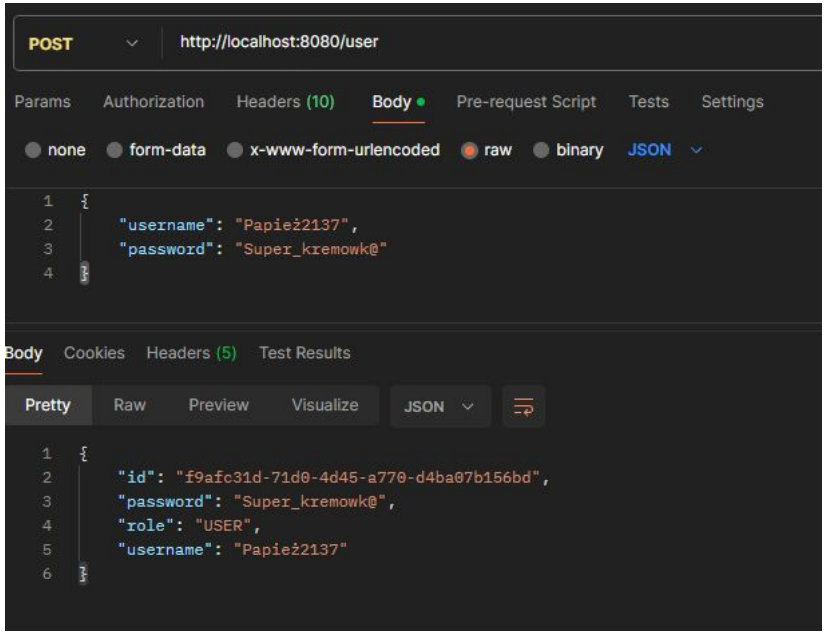
    @GetMapping
    public List<UserEntity> getAll() { return userService.getAll(); }

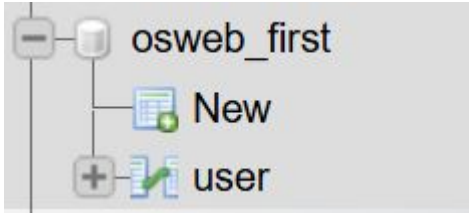
    @GetMapping("/{id}")
    public ResponseEntity<UserEntity> getById(@PathVariable String id){
        Optional<UserEntity> user = userService.getById(id);
        return user.map(ResponseEntity::ok).orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PutMapping("/{id}")
    public ResponseEntity<UserEntity> update(@PathVariable String id, @RequestBody UserDTO user){
        UserEntity updatedUser = userService.update(id, user);
        return ResponseEntity.ok(updatedUser);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<String> delete(@PathVariable String id){
        userService.delete(id);
        return ResponseEntity.ok( body: "User successfully deleted");
    }
}
```

Teraz możemy odpalić





Autoryzacja

Raczej nie chcemy żeby użytkownik mógł dla każdego zmienić username, lub by każdy mógł zmieniać rolę jak chce. Dlatego dodamy autoryzację przez JWT, gdzie niektóre funkcje będą dostępne od ręki, niektóre będą wymagać zalogowania, inne roli admina lub bycia danym użytkownikiem.

Użyjemy do tego bibliotek: java-jwt od auth0, spring-boot-starter-security i lombok

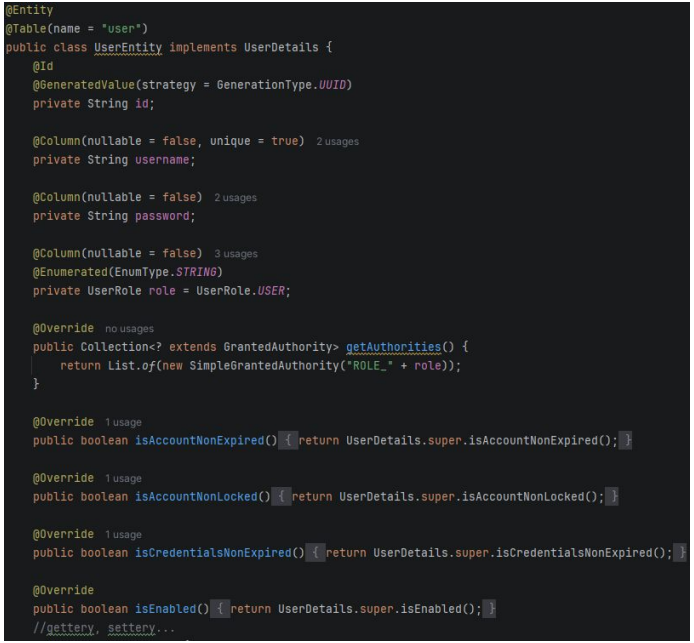
Stworzymy też nowy pakiet auth i tam będziemy tworzyć potrzebne nam pliki.

Krok 1: Dodajemy nowe paczki

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>com.auth0</groupId>
    <artifactId>java-jwt</artifactId>
    <version>4.4.0</version>
</dependency>
```

Krok 2: Przerobienie UserEntity

Implementujemy UserDetails które będzie nam służyło do autoryzacji użytkownika w spring boot.



```java
@Entity
@Table(name = "user")
public class UserEntity implements UserDetails {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @Column(nullable = false, unique = true)  2 usages
    private String username;

    @Column(nullable = false)  2 usages
    private String password;

    @Column(nullable = false)  3 usages
    @Enumerated(EnumType.STRING)
    private UserRole role = UserRole.USER;

    @Override  no usages
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority("ROLE_" + role));
    }

    @Override  1 usage
    public boolean isAccountNonExpired() { return UserDetails.super.isAccountNonExpired(); }

    @Override  1 usage
    public boolean isAccountNonLocked() { return UserDetails.super.isAccountNonLocked(); }

    @Override  1 usage
    public boolean isCredentialsNonExpired() { return UserDetails.super.isCredentialsNonExpired(); }

    @Override
    public boolean isEnabled() { return UserDetails.super.isEnabled(); }
    //gettery, settery...
```

Krok 3: Konfiguracja JWT

@Value - pobieramy zmienne środowiskowe z aplication.properties

Używamy klas JWT i Algorithm by stworzyć faktyczny obiekt JWT z odpowiednimi danymi, na określony czas.

```
jwt.secret=SuperNiesamowityKluczykImDluzsyTymLepszy
jwt.issuer=first-project
jwt.expiration-ms=3600000
```

```
@Service
public class AuthService {
    private final String issuer;  3 usages
    private final long expirationMs;  2 usages
    private final Algorithm algorithm;  3 usages

    public AuthService(  no usages
            @Value("${jwt.secret}") String secret,
            @Value("${jwt.issuer}") String issuer,
            @Value("${jwt.expiration-ms}") long expirationMs) {
        this.issuer = issuer;
        this.expirationMs = expirationMs;
        this.algorithm = Algorithm.HMAC256(secret);
    }

    public String generateToken(UserEntity user) {  no usages
        return JWT.create()
                .withSubject(user.getUsername())
                .withClaim( name: "id", user.getId())
                .withClaim( name: "role", user.getRole().ordinal())
                .withIssuer(issuer)
                .withExpiresAt(new Date(System.currentTimeMillis() + expirationMs))
                .sign(algorithm);
    }

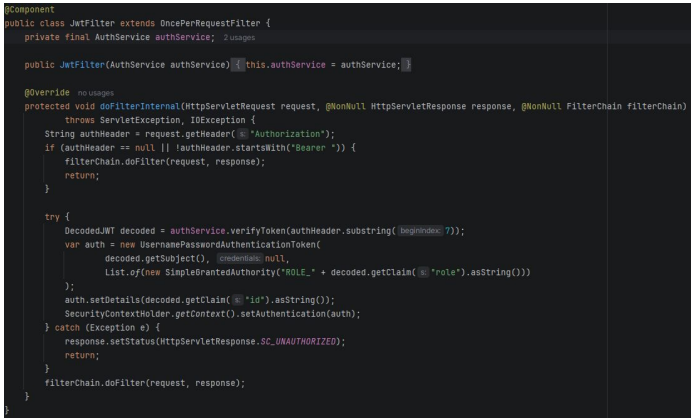
    public DecodedJWT verifyToken(String token) {  no usages
        return JWT.require(algorithm) Verification
                .withIssuer(issuer)
                .build() JWTVerifier
                .verify(token);
    }
}
```

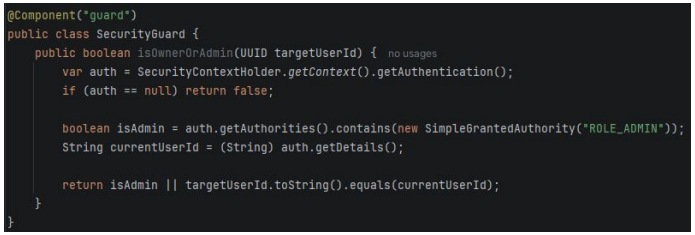
Krok 4: Filtr i Guard

Będą nam służyć do walidowania i zabezpieczania endpointów, tak by dostęp miał tylko admin i osoba której dana rzecz dotyczy.

@Component - oznaczenie jako komponent dla Springa

OncePerRequestFilter - interface określający filtr na każdy request, wykonujący się raz



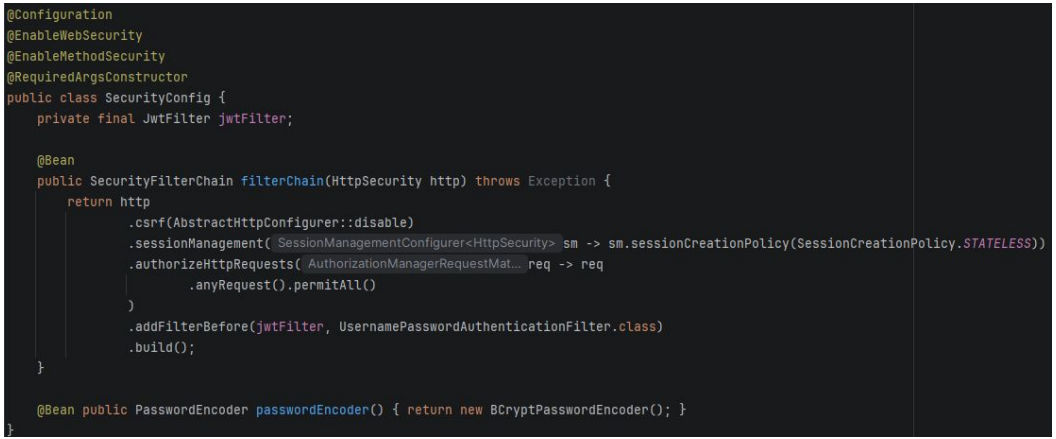


Krok 5: Konfiguracja security

@Configuration - flaga oznaczająca konfigurację

@EnableWebSecurity @EnableMethodSecurity - włączanie zabezpieczeń

@Bean - oznaczenie funkcji, by była flagą.



```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(AbstractHttpConfigurer::disable)
                .sessionManagement( SessionManagementConfigurer<HttpSecurity> sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests( AuthorizationManagerRequestMat... req -> req
                        .anyRequest().permitAll()
                )
                .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
                .build();
    }

    @Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

Krok 6: Dodajemy funkcję logowania

```java
public record LoginDTO(String username, String password) {}
public record AuthDTO(String token, String name) {}
```

```java
final UserRepo userRepo;
final AuthService authService;
private final PasswordEncoder passwordEncoder;

public AuthDTO login(LoginDTO request){  no usages
    UserEntity user = userRepo.findByUsername(request.username())
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));

    if (!passwordEncoder.matches(request.password(), user.getPassword())) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED);
    }

    String token = authService.generateToken(user);

    return new AuthDTO(token, user.getUsername());
}
```

```java
@Repository  2 usages
public interface UserRepo extends JpaRepository<UserEntity, String> {
    Optional<UserEntity> findByUsername(String username);  1 usage
}
```

```java
@RestController
@RequestMapping("/login")
@RequiredArgsConstructor
public class LoginController {
    private final UserService userService;

    @PostMapping
    public AuthDTO login(@RequestBody LoginDTO request){
        return userService.login(request);
    }
}
```

Krok 7: Dodanie flag

@PreAuthorize - określa politykę wobec endpointu

My używamy permitAll, hasRole i isAuthenticated kiedy potrzebujemy oraz @guard.isOwnerOrAdmin zdefiniowany wcześniej.

```java
@PostMapping
@PreAuthorize("permitAll()")
public ResponseEntity<UserEntity> create(@RequestBody UserDTO user){
    UserEntity newUser = userService.create(user);
    return ResponseEntity.ok(newUser);
}

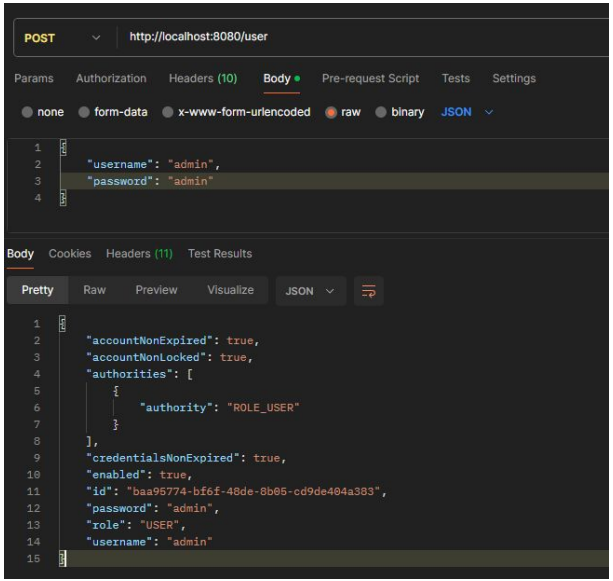
@GetMapping
@PreAuthorize("hasRole('ADMIN')")
public List<UserEntity> getAll() { return userService.getAll(); }

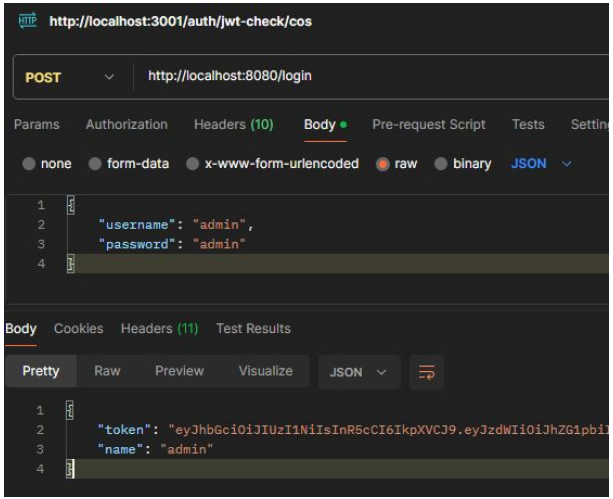
@GetMapping("/{id}")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserEntity> getById(@PathVariable String id){
    Optional<UserEntity> user = userService.getById(id);
    return user.map(ResponseEntity::ok).orElseGet(() -> ResponseEntity.notFound().build());
}

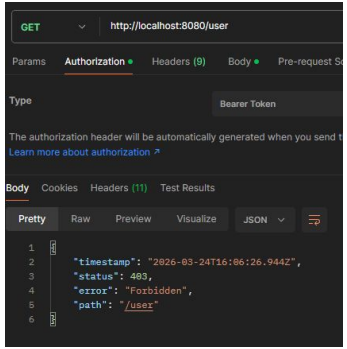
@PutMapping("/{id}")
@PreAuthorize("@guard.isOwnerOrAdmin(#id)")
public ResponseEntity<UserEntity> update(@PathVariable String id, @RequestBody UserDTO user){
    UserEntity updatedUser = userService.update(id, user);
    return ResponseEntity.ok(updatedUser);
}

@DeleteMapping("/{id}")
@PreAuthorize("@guard.isOwnerOrAdmin(#id)")
public ResponseEntity<String> delete(@PathVariable String id){
    userService.delete(id);
    return ResponseEntity.ok( body: "User successfully deleted");
}
```

Sprawdzamy czy działa







WebSockets

Do stworzenia chatu, lub poprostu dynamicznie zmieniających się danych, przyda nam się WebSockets. W tym celu stworzymy sobie prosty Controller i skonfugurujemy STOMP, by zrobić sobie prosty Websockets, bez niczego wybitnego.

Krok 1: dodajemy paczki

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

Krok 2: Stworzenie pliku konfiguracji

Będzie nam służyć za prosty STOMP Broker.
WebSocketMessageBrokerConfigurer - interface do tworzenia brokera

configureMessageBroker - tworzy nam broker i tworzy miejsce na które ma odpowiadać websocket

registerStompEndpoint - tworzy nam endpoint do połączenia z backendem

```
@Configuration  no usages  new *
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override  no usages  new *
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker( ...destinationPrefixes: "/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override  no usages  new *
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint( ...paths: "/gs-guide-websocket");
    }
}
```

Krok 3: Stworzenie kontrolera

Tutaj tworzymy faktyczny endpoint i miejsce do którego zwracamy wiadomość

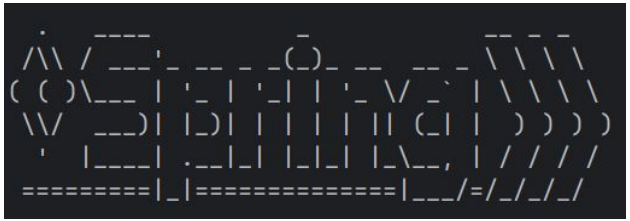
@MessageMapping - miejsce nasłuchujące event

@SentTo - odpowiedź do każdego przypisanego do danego endpointu

```
@Controller  no usages  new *
public class GreetingController {
    @MessageMapping("/hello")  no usages  new *
    @SendTo("/topic/greetings")
    public String greeting(String message) throws Exception {
        Thread.sleep( millis: 1000);
        return "hi " + message;
    }
}
```

Teraz możemy sprawdzić

Teoretycznie działa? Trzeba by było stworzyć Frontend



Źródła:

- Gemini
- Dokumentacja Java Spring Boot
- dev.to